SmartCampus Solutions

Memoria técnica de la práctica de laboratorio

Despliegue de una aplicación con Docker y Kubernetes

Jaime Mas
Jaime Moraleda
Mario

Grado en Ingeniería Informática · Universidad Camilo José Cela

2 de noviembre de 2026

Repositorio: Repositorio de Jaime Mas, Jaime Moraleda y Mario

Aplicación multicontenedor con frontend, API REST y PostgreSQL. Despliegue local con Docker Compose y Minikube.

1. Introducción y alcance

SmartCampus Solutions necesita un prototipo para consultar aulas universitarias y gestionar sus reservas. El trabajo separa la interfaz web, la lógica de negocio y el almacenamiento en tres servicios independientes, y despliega la misma aplicación con Docker Compose y Kubernetes.

Nuestro objetivo ha sido comprender la construcción de imágenes, la comunicación entre servicios, la persistencia y las diferencias de operación entre ambos entornos. En Kubernetes hemos probado el escalado del backend, la sustitución de un Pod y la modificación de configuración.

Aplicación desarrollada

La interfaz muestra aulas, capacidad y equipamiento; permite consultar, crear y eliminar reservas. La API también permite crear y eliminar aulas. PostgreSQL almacena las tablas rooms y bookings, relacionadas mediante aula_id. Los datos iniciales incluyen tres aulas y dos reservas.

La API valida los tipos de entrada y que la hora de fin sea posterior a la inicial. Las consultas SQL usan parámetros. Nuestro prototipo no incorpora autenticación ni comprobación de reservas solapadas; todavía no está preparado para producción.

Organización del trabajo y herramientas

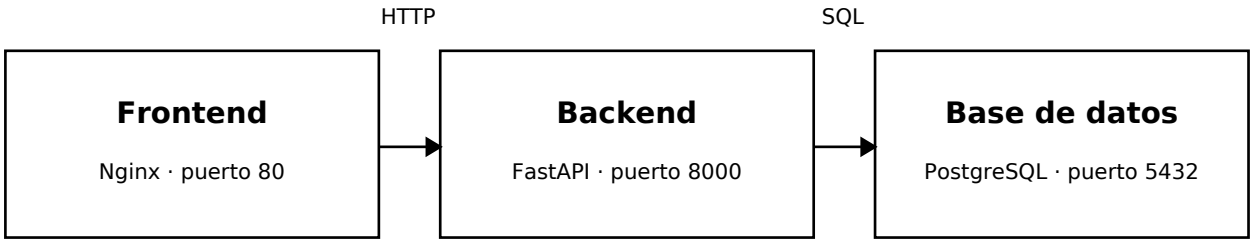
Dividimos el trabajo en backend y base de datos, frontend y Docker Compose, y despliegue Kubernetes. Utilizamos ramas y Pull Requests para integrar los cambios en main. Realizamos el desarrollo y ejecutamos los comandos con Visual Studio Code y PowerShell en Windows.

El entorno documentado utiliza Docker 29.8.0, Compose v5.5.1, Minikube v1.39.0 y kubectl v1.36.1. El clúster smartcampus ejecuta Kubernetes v1.35.0 con el driver Docker, dos CPU y 2200 MB de memoria asignada.

Lectura de las evidencias

Las figuras recogen las capturas que realizamos durante las pruebas. Hemos seleccionado las más representativas y eliminado los duplicados. La persistencia inicial está documentada con contenedores Docker independientes; no guardamos una captura específica equivalente de Compose. Tampoco guardamos una captura de la web durante la parada del backend ni del valor anterior de APP_ENV tras actualizar el ConfigMap. Conservamos estas limitaciones en la documentación para distinguir los resultados registrados de la explicación técnica.

2. Arquitectura y componentes



El navegador descarga los archivos del frontend y solicita /api/rooms y /api/bookings al mismo origen. Nginx elimina el prefijo /api/ y reenvía la petición a FastAPI. El backend utiliza Psycopg para consultar PostgreSQL. El navegador no intenta resolver nombres internos del clúster.

Componente	Compose	Kubernetes
Frontend	Servicio frontend; host 8080 → 80	Deployment frontend; frontend-service NodePort 30080
Backend	Servicio backend; host 8000 → 8000	Deployment backend; 2 réplicas; backend-service:8000
PostgreSQL	Servicio database; volumen database-data	StatefulSet database; database-service; PVC de 1 Gi
Configuración	Variables desde .env; archivo ignorado por Git	smartcampus-config y database-secret

API REST

GET /health comprueba la respuesta del proceso y muestra APP_ENV. GET /ready consulta PostgreSQL para verificar disponibilidad. Los endpoints obligatorios son GET /rooms, GET /rooms/{id} y GET /bookings, además de /health. Añadimos POST y DELETE para aulas y reservas. /docs ofrece la documentación interactiva de FastAPI.

Datos y estado

La API no mantiene sesiones ni datos de negocio en memoria local, por lo que sus réplicas son intercambiables. PostgreSQL conserva el estado y necesita almacenamiento persistente. init.sql solo se ejecuta automáticamente al inicializar un directorio de datos vacío.

3. Despliegue con Docker Compose

Imágenes y Dockerfiles

El backend parte de python:3.11-slim. Copia requirements.txt, instala FastAPI, Uvicorn y Psycopg, copia la aplicación y ejecuta Uvicorn en 0.0.0.0:8000 con un usuario sin privilegios. .dockerignore excluye cachés y configuración local. La imagen se etiqueta smartcampus-backend:1.0.

El frontend utiliza nginx:stable-alpine. Copia HTML, CSS y JavaScript al directorio público y una plantilla a /etc/nginx/templates/default.conf.template. Al arrancar, Nginx recibe BACKEND_URL y genera la configuración del proxy. PostgreSQL utiliza la imagen oficial postgres:17.

Red, puertos y configuración

docker-compose.yml define frontend, backend y database en la red bridge campus-net. Los nombres de los servicios actúan como nombres DNS internos. El backend recibe DB_HOST=database y el frontend BACKEND_URL=http://backend:8000. PostgreSQL no publica un puerto en el host; frontend y backend publican 8080 y 8000 respectivamente.

Las credenciales se leen de .env, excluido del repositorio; .env.example contiene valores de ejemplo. Esto evita incluir la contraseña en el código, aunque las variables de entorno siguen requiriendo protección y .env no es un almacén cifrado.

Dependencias y persistencia

El backend espera a que PostgreSQL supere pg_isready. El frontend espera a que /ready del backend sea satisfactorio. depends_on ordena el arranque, pero no sustituye una recuperación continua ante fallos. Los healthchecks indican salud; por sí solos no reinician un contenedor no saludable.

El volumen database-data conserva /var/lib/postgresql/data. init.sql se monta en modo lectura. docker compose down conserva el volumen; down -v lo elimina. Cambiar POSTGRES_PASSWORD en .env no modifica automáticamente la contraseña de una base de datos ya creada.

Ejecución reproducible desde la raíz

```
Copy-Item .env.example .env

# Editar las dos contraseñas de ejemplo con el mismo valor.

docker compose up -d --build

docker compose ps

docker compose logs --tail 60
```

La interfaz se abre en http://localhost:8080 y la documentación de la API en http://localhost:8000/docs. Deben estar libres los puertos publicados. La configuración no incluye restart, para observar la parada manual sin recuperación automática configurada.

4. Despliegue con Kubernetes

Construimos las mismas imágenes del frontend y backend y las cargamos con minikube image load en el perfil smartcampus. imagePullPolicy: IfNotPresent permite utilizar esas imágenes locales. Todos los recursos de la aplicación se agrupan en el Namespace smartcampus.

Deployments y comprobaciones

El backend se ejecuta en un Deployment con dos réplicas y límites de CPU y memoria. startupProbe permite su arranque; readinessProbe consulta /ready para retirar de los endpoints un Pod que no pueda atender peticiones; livenessProbe consulta /health para detectar un proceso que no responda. No se usa la base de datos como condición de liveness para evitar reinicios innecesarios de la API ante una caída de PostgreSQL.

El frontend utiliza otro Deployment con una réplica y probes HTTP sobre /. BACKEND_URL llega desde el ConfigMap. Su disponibilidad HTTP no implica por sí sola que toda la cadena hasta PostgreSQL funcione; la prueba de la web con datos verifica la integración.

Services y exposición

backend-service es ClusterIP y selecciona los Pods app=backend. frontend-service es NodePort y publica el puerto 30080 del nodo hacia el 80 del contenedor. En Windows se accede mediante minikube service, que puede mantener un túnel mientras su terminal permanezca abierta.

database-service es headless (clusterIP: None): resuelve los endpoints del StatefulSet en lugar de ofrecer una IP virtual. La aplicación utiliza su nombre DNS, no una IP de Pod escrita en el código. Aquí solo existe una instancia de PostgreSQL.

StatefulSet y volumen

Elegimos StatefulSet para PostgreSQL por su identidad estable database-0 y la asociación estable con un PVC mediante volumeClaimTemplates. El volumen solicita 1 Gi con ReadWriteOnce y la StorageClass standard de Minikube. Reemplazar el Pod no equivale a borrar su PVC. Un StatefulSet no configura por sí solo replicación, copias de seguridad ni alta disponibilidad de PostgreSQL.

ConfigMap y Secret

smartcampus-config contiene host, puerto y nombre de base de datos, APP_ENV y BACKEND_URL. database-secret se crea por comando y contiene DB_USER y DB_PASSWORD; no se versiona una contraseña real. El SQL inicial se distribuye mediante database-init, guardado como manifiesto para reproducir el despliegue.

Los Secrets separan los datos sensibles y permiten controlar su acceso con RBAC. Base64 es codificación, no cifrado; deben configurarse cifrado en reposo y permisos adecuados. Un gestor especializado añade funciones como rotación y auditoría. Los cambios de variables de entorno requieren sustituir los Pods consumidores [2].

5. Reproducción del despliegue Kubernetes

Ejecutamos los comandos desde la raíz del repositorio, con Docker Desktop iniciado. En un entorno nuevo hay que crear primero el Namespace y el Secret. La contraseña de ejemplo debe sustituirse antes de ejecutar el comando.

```
minikube start -p smartcampus --driver=docker --cpus=2 --memory=2200 --kubernetes-version=v1.35.0

docker build -t smartcampus-backend:1.0 ./backend

docker build -t smartcampus-frontend:1.0 ./frontend

minikube image load smartcampus-backend:1.0 -p smartcampus

minikube image load smartcampus-frontend:1.0 -p smartcampus

kubectl --context=smartcampus apply -f k8s/namespace.yaml
```

Crear database-secret con kubectl create secret generic en el Namespace smartcampus y las claves DB_USER y DB_PASSWORD, como se explica en docs/kubernetes.md. No incluimos aquí una credencial real.

```
kubectl --context=smartcampus apply -f k8s/

kubectl --context=smartcampus -n smartcampus get pods

kubectl --context=smartcampus -n smartcampus get services

kubectl --context=smartcampus -n smartcampus get deployments

minikube service frontend-service -n smartcampus -p smartcampus
```

Operaciones probadas

```
kubectl -n smartcampus scale deployment backend --replicas=4

kubectl -n smartcampus scale deployment backend --replicas=2

kubectl -n smartcampus delete pod NOMBRE_DEL_POD

kubectl -n smartcampus get pods -l app=backend -w

kubectl apply -f k8s/configmap.yaml

kubectl -n smartcampus rollout restart deployment/backend
```

En este último bloque utilizamos el contexto smartcampus. Para evitar actuar sobre otro entorno puede añadirse --context=smartcampus a cada comando, como en las pruebas realizadas.

El autoscaling con HPA y la exposición mediante Ingress son ampliaciones opcionales y no forman parte del despliegue realizado. Las instrucciones detalladas, esperas de rollout y diagnóstico están en docs/kubernetes.md.

6. Pruebas: Docker y API

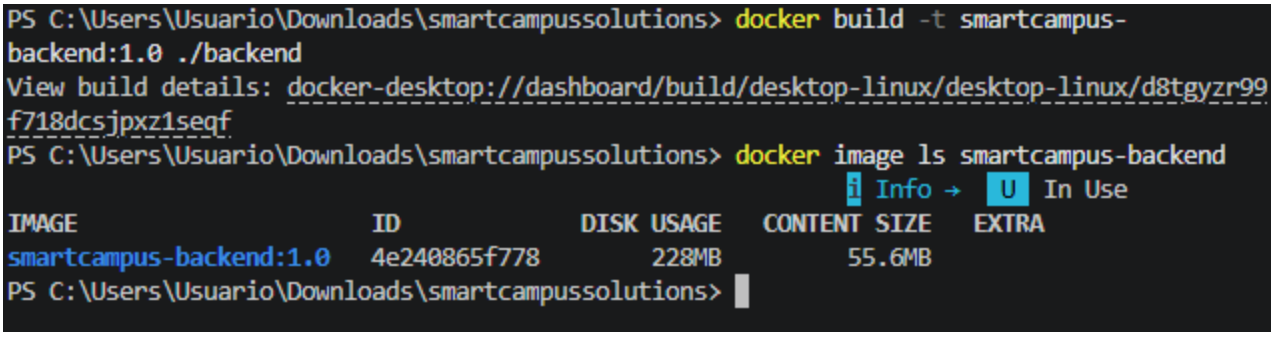


Figura 1. Construcción de la imagen y listado de smartcampus-backend:1.0.

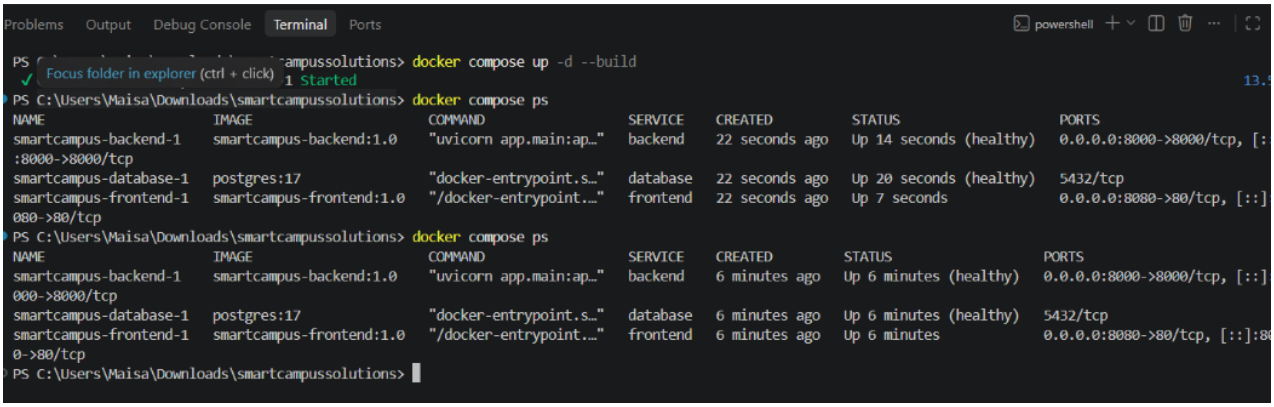


Figura 2. docker compose ps muestra los tres servicios activos; backend y PostgreSQL aparecen saludables.



Figura 3. /ready devuelve database: connected, verificando la comunicación API-PostgreSQL.

La construcción deja una imagen identificable y Compose inicia los tres servicios. El endpoint /ready verifica una consulta a PostgreSQL, mientras que /health comprueba el proceso de la API. Son comprobaciones complementarias.

7. Pruebas: aplicación y persistencia inicial

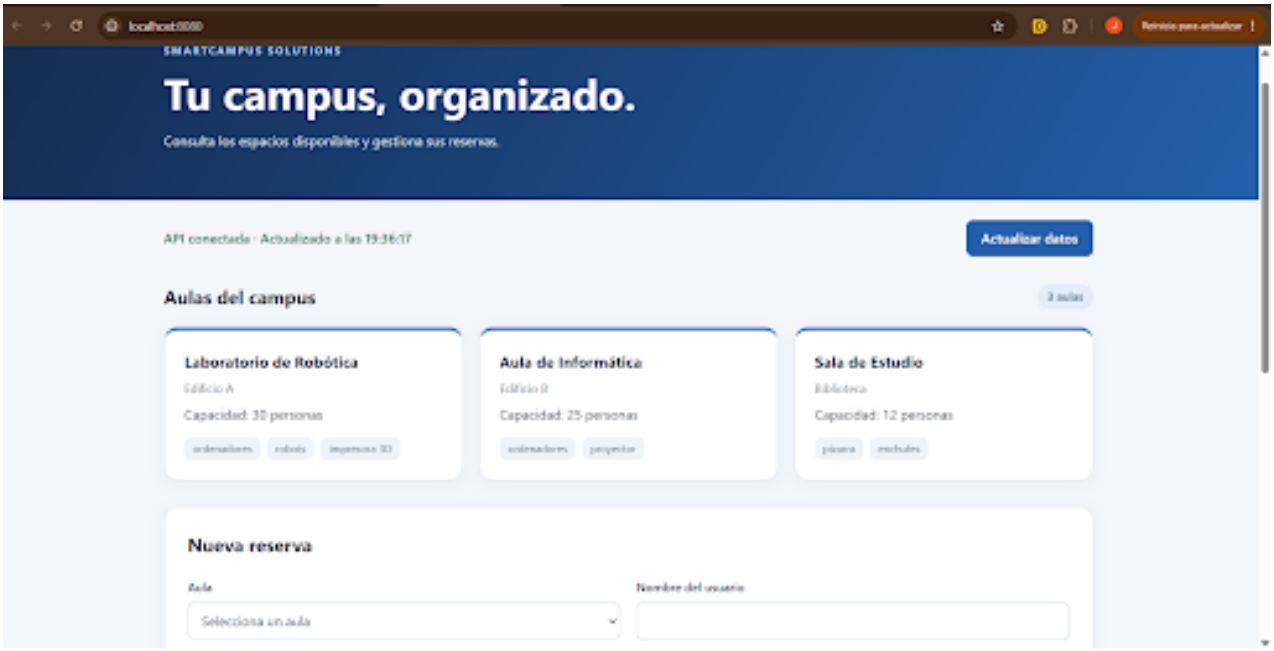


Figura 4. Aplicación con Compose: API conectada y tres aulas obtenidas del backend.

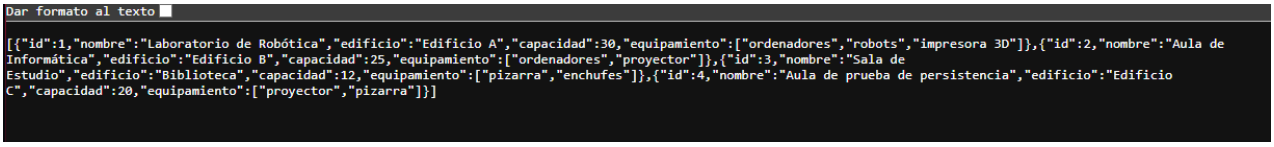


Figura 5. Consulta con cuatro aulas antes del reinicio, incluida Aula de prueba de persistencia.

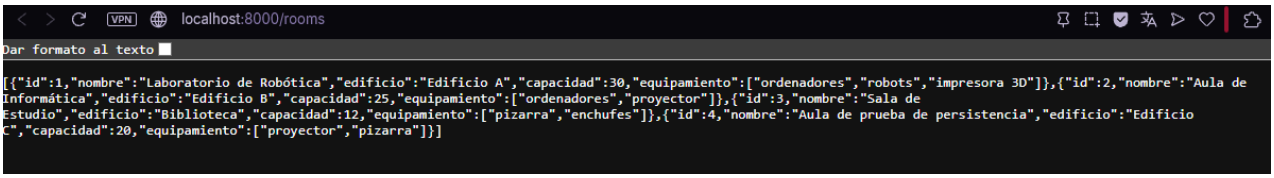


Figura 6. El aula de prueba continúa en la respuesta después del reinicio.

Las figuras 5 y 6 corresponden a la prueba inicial con Docker: los datos continúan tras reiniciar los contenedores. No documentan por sí solas la recreación con Compose. Este utiliza un volumen persistente en su configuración, pero no guardamos una captura específica del antes y después de down/up.

8. Pruebas: parada y recursos Kubernetes



Figura 7. docker compose stop backend deja el backend en Exited; los demás servicios siguen activos.

No configuramos una política de reinicio. Una parada manual no se recupera inmediatamente por activar una política restart: Docker trata específicamente las paradas manuales [1]. La recuperación prevista es docker compose start backend. No guardamos una captura de la web durante el error ni de su recuperación.

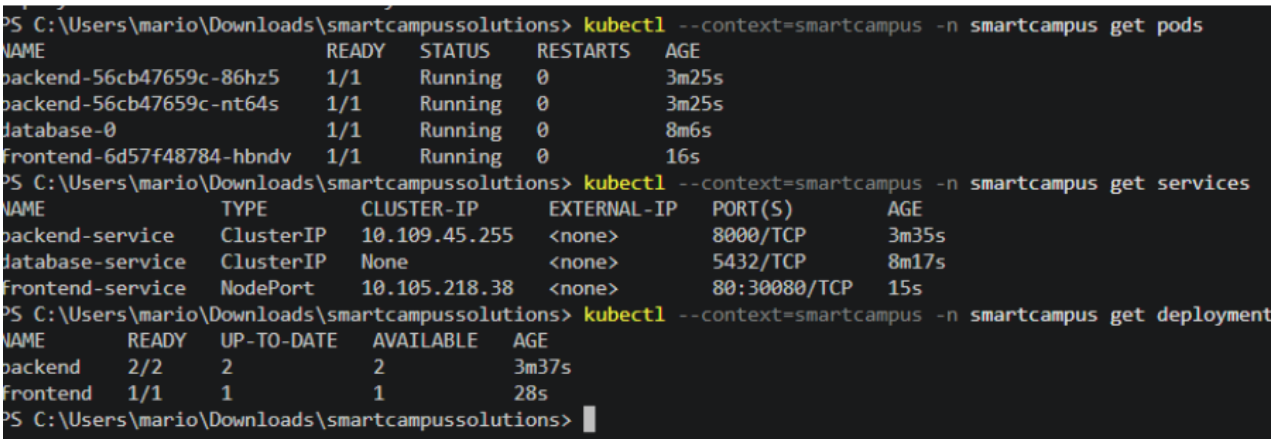


Figura 8. Kubernetes: cuatro Pods Running, tres Services y Deployments con backend 2/2 y frontend 1/1.

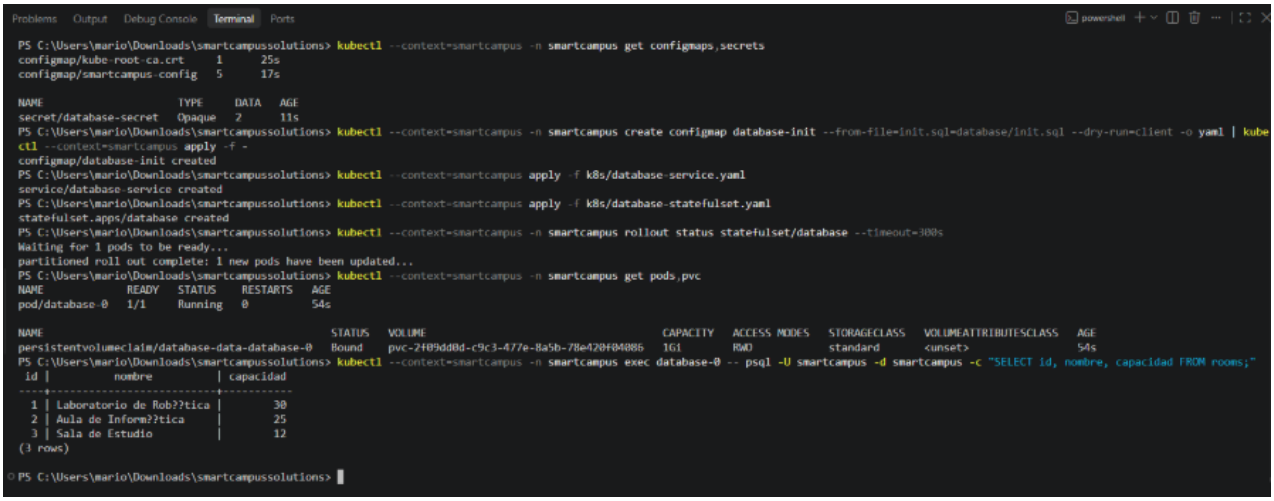


Figura 9. PostgreSQL database-0 Running, PVC Bound y consulta de las aulas iniciales.

9. Pruebas: web, escalado y recuperación

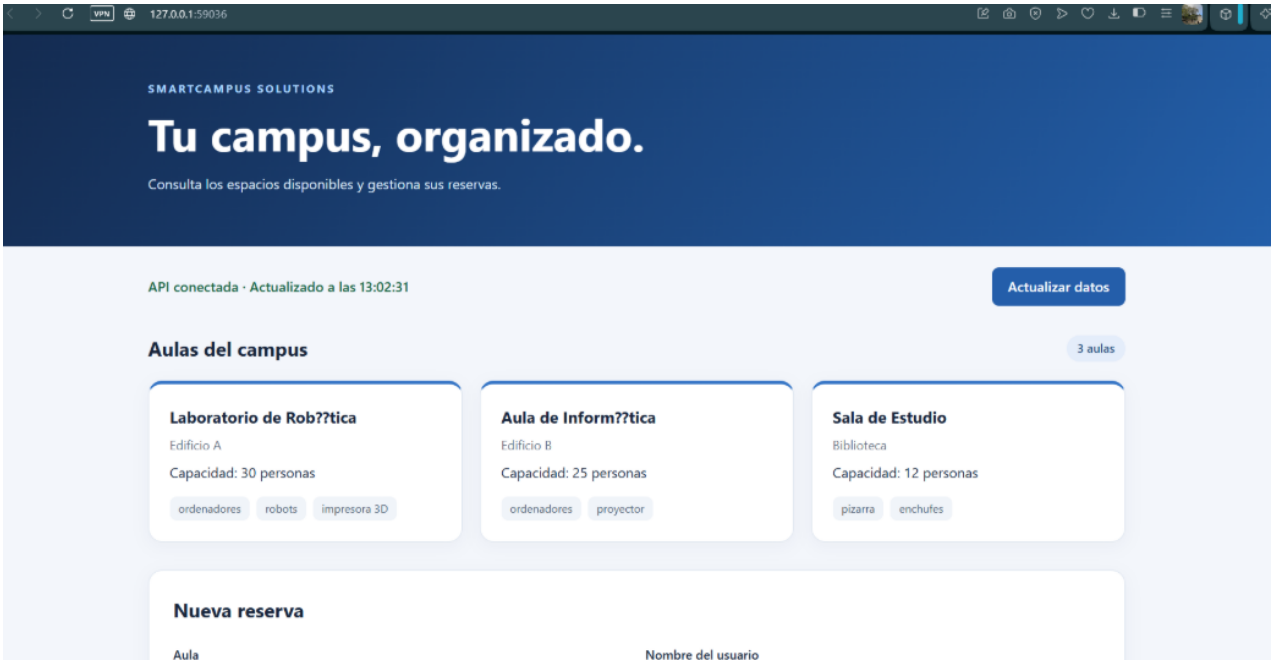


Figura 10. Aplicación accesible desde fuera del clúster mediante el acceso generado por Minikube.

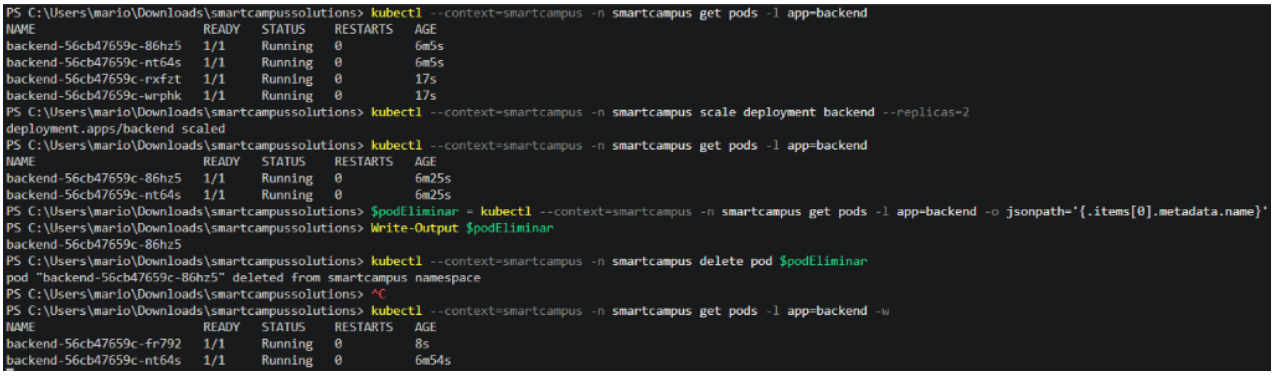


Figura 11. Cuatro réplicas, reducción a dos y eliminación del Pod backend-56cb47659c-86hz5. Aparece el sustituto backend-56cb47659c-fr792.

El listado final muestra dos backends disponibles. El Pod nuevo tiene una edad menor y un nombre distinto. El ReplicaSet, gestionado por el Deployment, crea el sustituto para mantener las réplicas deseadas. Esto no equivale a reiniciar el contenedor eliminado.

La web presenta caracteres acentuados incorrectos en algunos datos del entorno Kubernetes. Lo dejamos registrado como un defecto pendiente de revisar; no impide verificar la comunicación, pero debe corregirse para una presentación final sin errores de codificación.

10. Pruebas: actualización del ConfigMap

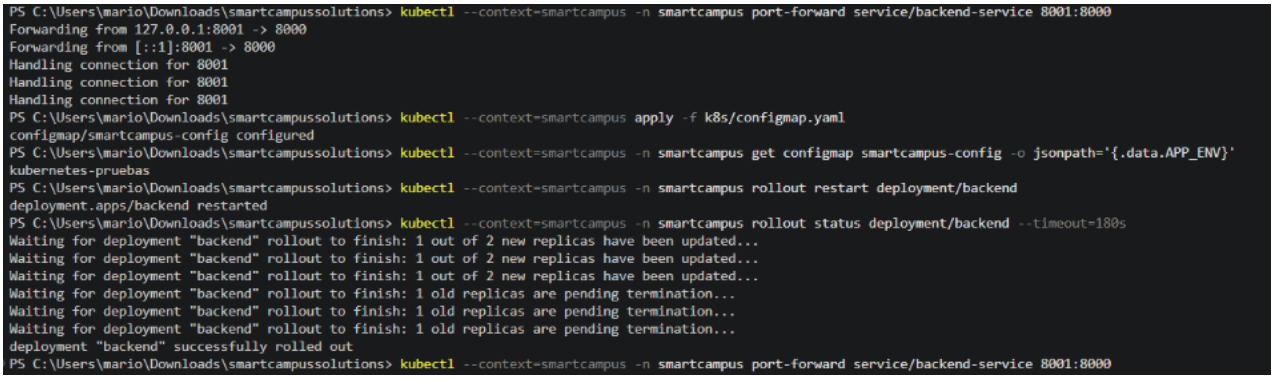


Figura 12. Aplicación del ConfigMap, lectura de APP_ENV y rollout restart del backend.

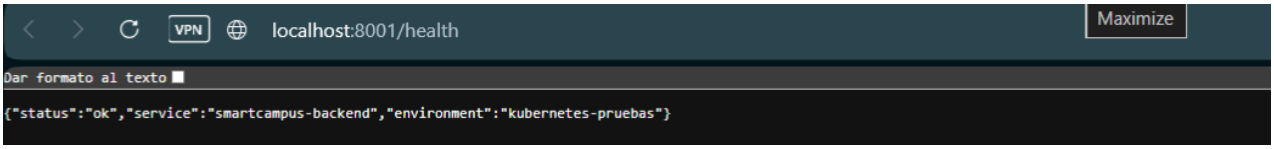


Figura 13. /health devuelve environment: kubernetes-pruebas después de la sustitución de Pods.

Modificamos APP_ENV de kubernetes a kubernetes-pruebas y aplicamos el ConfigMap. Los valores inyectados como variables de entorno no cambian en procesos existentes; los nuevos Pods reciben la configuración tras el rollout [2].

La evidencia disponible muestra la actualización y el resultado final. No guardamos una captura del valor anterior después de aplicar el ConfigMap y antes del reinicio. Por ello explicamos ese estado intermedio a partir del comportamiento de las variables de entorno, aunque no contamos con su captura.

Balance de las pruebas

Hemos documentado el despliegue Compose, estado de API y base de datos, despliegue Kubernetes, volumen persistente, exposición del frontend, escalado, sustitución de Pod y actualización final de configuración. No realizamos pruebas de rendimiento ni de fallo de un nodo completo.

11. Comparativa técnica

Característica	Docker Compose	Kubernetes
Ejecución local	Arranque sencillo en un host Docker.	Requiere clúster; Minikube permite practicar localmente.
Orquestación	Define servicios, redes, volúmenes y dependencias en un host.	Controladores reconcilian recursos y planifican Pods en nodos.
Escalado	Manual con up --scale; necesita evitar puertos host fijos repetidos.	Réplicas en Deployment; HPA opcional con métricas.
Self-healing	Políticas restart para salidas; no repone por sí solo contenedores eliminados.	ReplicaSet sustituye Pods; kubelet reinicia contenedores según política.
Descubrimiento	DNS por nombre de servicio en la red.	DNS de Services; endpoints actualizados según Pods disponibles.
Configuración	Variables, env_file y archivos montados.	ConfigMaps y referencias declarativas.
Secrets	Montaje de secretos desde fuentes configuradas; .env no es un gestor de secretos.	Objetos Secret con RBAC; requieren protección y cifrado adecuados.
Alta disponibilidad	Limitada al host del despliegue.	Posible con varios nodos y réplicas; no queda garantizada por usar Kubernetes.
Balanceo	Hay que diseñar proxy o balanceador; DNS no basta para todas las necesidades.	Services ClusterIP distribuyen conexiones a endpoints preparados.
Actualizaciones	Reconstrucción y recreación de servicios.	Deployment permite rolling updates y rollback.
Complejidad	Menos recursos y configuración.	Más componentes, manifiestos y operación.
Uso habitual	Desarrollo, pruebas y aplicaciones pequeñas en un host.	Aplicaciones con varios servicios, escalado y operación en clúster.

En SmartCampus, Compose facilita reproducir el entorno de desarrollo. Kubernetes aporta reconciliación y operación por réplicas, a cambio de mayor complejidad. El clúster local de un nodo y PostgreSQL único no son una arquitectura de alta disponibilidad.

12. Preguntas obligatorias (1-7)

1. ¿Qué diferencia hay entre un contenedor Docker y un Pod?

Un contenedor ejecuta un proceso aislado a partir de una imagen. Un Pod es la unidad de ejecución y planificación de Kubernetes: agrupa uno o más contenedores que comparten red y pueden compartir volúmenes. Kubernetes puede usar runtimes distintos de Docker Engine.

2. ¿Qué problema resuelve Docker Compose?

Describe varios servicios y sus redes, volúmenes y configuración en un archivo. Permite arrancarlos de forma coordinada sin repetir comandos docker run para cada componente.

3. ¿Qué problemas adicionales resuelve Kubernetes?

Gestiona planificación en nodos, réplicas, sustitución de Pods, descubrimiento, exposición y actualizaciones declarativas. Añade capacidades de escalado y operación continua que Compose no ofrece como reconciliación de clúster.

4. ¿Qué ocurre al eliminar un contenedor de Compose?

Compose no mantiene un controlador que lo reponga continuamente. Una política restart tampoco puede recuperar un contenedor eliminado. docker compose up -d puede crearlo de nuevo. Detenerlo es diferente de eliminarlo: en la prueba se utilizó stop.

5. ¿Qué ocurre al eliminar un Pod de un Deployment?

Su ReplicaSet detecta que faltan réplicas y crea un Pod sustituto. El reemplazo tiene identidad distinta y requiere tiempo para arrancar y superar readiness.

6. ¿Cómo descubre servicios Kubernetes?

El DNS del clúster resuelve nombres de Services. Los selectores y EndpointSlices relacionan esos Services con los Pods disponibles. Un Service normal ofrece una IP virtual; uno headless devuelve los endpoints.

7. ¿Por qué evitar una IP de Pod escrita directamente?

La IP puede cambiar al sustituir el Pod. Un nombre de Service evita acoplar la configuración a una instancia y permite mantener el acceso cuando cambian los endpoints.

13. Preguntas obligatorias (8-14)

8. ¿Qué función cumple un Deployment?

Declara el estado deseado de réplicas y de la plantilla de Pods. Gestiona ReplicaSets para aplicar actualizaciones y mantener las instancias previstas.

9. ¿Qué función cumple un Service?

Proporciona un punto de descubrimiento y acceso a un conjunto de Pods. Un ClusterIP distribuye conexiones entre endpoints; NodePort permite acceso mediante puertos de los nodos. Un Service headless no utiliza esa IP virtual.

10. ¿Deployment frente a StatefulSet?

Deployment está orientado a réplicas intercambiables. StatefulSet aporta identidad estable, orden de operación y asociación con volúmenes por réplica. No implementa por sí mismo la replicación interna de una base de datos.

11. ¿Por qué tratar una base de datos de otra forma?

Contiene estado que debe sobrevivir al reemplazo de procesos. Necesita integridad, almacenamiento, copias y recuperación. Aumentar réplicas de PostgreSQL sin configurar replicación no crea una base coherente compartida.

12. ¿Qué solucionan ConfigMap y Secret?

Separan configuración y credenciales del código y de la imagen. ConfigMap almacena datos no sensibles; Secret permite gestionar datos sensibles con controles específicos. Ninguno sustituye una política de seguridad y rotación.

13. ¿Cómo se escala con Compose?

Por ejemplo, docker compose up -d --scale backend=4. En este proyecto hay que modificar antes la publicación fija 8000:8000 para evitar conflictos entre réplicas y configurar adecuadamente el acceso del proxy. No realizamos esta ampliación de Compose.

14. ¿Cómo se escala con Kubernetes?

Se modifica replicas en el manifiesto o se ejecuta kubectl scale deployment backend --replicas=4 en el Namespace correspondiente. El controlador crea los Pods necesarios. Después se redujo a dos; HPA sería una ampliación distinta.

14. Preguntas obligatorias (15-20)

15. ¿Qué sucede cuando falla un Pod?

Depende del fallo: kubelet puede reiniciar un contenedor en el mismo Pod; si el Pod se elimina o falla de forma terminal, su controlador puede crear otro. Readiness retira endpoints no preparados. Un Pod aislado sin controlador no tiene garantizada su sustitución.

16. ¿Qué ventajas ofrece para múltiples servicios?

Centraliza despliegue declarativo, descubrimiento, límites de recursos, escalado por componente y actualizaciones. Los controladores reducen intervención manual al mantener el estado deseado.

17. ¿Qué inconvenientes introduce frente a Compose?

Consume más recursos y requiere aprender red, almacenamiento, permisos, recursos y diagnóstico del clúster. Para un prototipo pequeño, el esfuerzo de operación puede superar las ventajas.

18. ¿Cuándo utilizar Compose?

En desarrollo local, pruebas de integración o un proyecto pequeño en un único servidor, cuando prima un entorno reproducible con baja complejidad y se aceptan sus límites de disponibilidad.

19. ¿Cuándo utilizar Kubernetes?

Cuando varios servicios necesitan escalado independiente, despliegues frecuentes y recuperación gestionada sobre varios nodos, y el equipo dispone de capacidad para operar el clúster o utilizar un servicio gestionado.

20. ¿Qué falta para llevarlo a cloud?

Publicar imágenes versionadas en un registro y configurar credenciales de descarga; desplegar un clúster y almacenamiento duradero adecuados; configurar dominio, TLS y entrada externa. Añadir autenticación, permisos, secretos gestionados, copias de PostgreSQL y recuperación probada, observabilidad, CI/CD y límites de red. Dimensionar costes y recursos. Para disponibilidad real se necesitan varios nodos y una estrategia de base de datos replicada o gestionada.

15. Problemas, conclusiones y referencias

Incidencias reales y resolución

Motor Docker no iniciado. docker info mostraba un error de conexión al pipe dockerDesktopLinuxEngine. Iniciamos Docker Desktop y esperamos a que el motor estuviera disponible. Después comprobamos la información de Server y construimos la imagen correctamente. No encontramos errores adicionales en el Dockerfile.

Compose no encontraba configuración. El comando devolvió no configuration file provided. Creamos y guardamos docker-compose.yml en la raíz y lo validamos con docker compose config --quiet. Después comprobamos que los servicios se comunicaban correctamente. No detectamos un problema adicional de red.

Perfil Minikube incorrecto. Escribimos smartcampus+ por error al cargar la imagen. Corregimos el nombre a smartcampus y comprobamos que ambas imágenes aparecían en el listado del clúster.

Permisos de GitHub. Mario recibió un error 403 al intentar subir su rama. Jaime Mas envió la invitación de colaborador y Mario la aceptó. Repetimos el push y terminó correctamente. También configuramos el nombre y correo de Git para identificar la autoría de los commits.

Caracteres acentuados. La web de Kubernetes muestra texto incorrecto en algunos nombres. Nos queda pendiente comprobar la codificación UTF-8 del SQL y del ConfigMap y aplicar una corrección controlada de los datos existentes. Regenerar el SQL inicial no modifica una base ya inicializada. Esta incidencia no quedó resuelta durante las pruebas.

Conclusiones

Para desarrollo local y pruebas de integración elegiríamos Compose: reduce los pasos de arranque y reproduce las dependencias. Para un proyecto personal pequeño en un servidor también puede ser suficiente, acompañándolo de copias, seguridad y supervisión.

Para una aplicación empresarial con múltiples servicios y necesidad de escalado elegiríamos Kubernetes si existe capacidad de operación: permite ajustar réplicas y reemplazar Pods mediante controladores. Las pruebas mostraron estas funciones, pero no acreditan alta disponibilidad del sistema completo. El almacenamiento y la base de datos requieren una estrategia propia.

La aplicación mantiene la misma separación de componentes en ambos despliegues. La elección depende de tamaño, disponibilidad requerida y coste operativo; Kubernetes no elimina la necesidad de diseñar y probar la aplicación ni la recuperación de sus datos.

Referencias y material del proyecto

[1] Docker Docs. Start containers automatically.
https://docs.docker.com/engine/containers/start-containers-automatically/

[2] Kubernetes. Updating Configuration via a ConfigMap.
https://kubernetes.io/docs/tutorials/configuration/updating-configuration-via-a-configmap/

Código y manifiestos: Repositorio de Jaime Mas, Jaime Moraleda y Mario
Instrucciones: README.md, docs/backend.md y docs/kubernetes.md.
Enunciado de la práctica y capturas de nuestras pruebas. Consulta de referencias: 2 de noviembre de 2026.